



Figure 1: The more complex directed graph *Dgraph_12*

The code does not need any explanation, as it is similar to the directed graphs we have generated in the previous article in this series. Upon executing the program ‘*dgraph12.sagews*’ using CoCalc, we obtain the graph called *Dgraph_12*, which is shown in Figure 1.

Now, let us see how the adjacency matrix of a directed graph looks. To do this, add the following line of code at the end of the program called ‘*dgraph12.sagews*’.

```
DG.adjacency_matrix( )
```

Upon executing the modified program, the following adjacency matrix will also be displayed on the screen.

```
[0 1 0 0 1 0]
[0 0 1 0 1 0]
[0 0 0 1 0 0]
[0 1 0 0 1 0]
[0 0 0 0 0 1]
[1 0 0 0 0 0]
```

The adjacency matrix shown above represents a directed graph with 6 vertices and, hence, is a 6 x 6 matrix. Each entry at position (i, j) in the matrix indicates whether there is a directed edge from vertex *i* to vertex *j*. An entry of 1 indicates that there is a directed edge from vertex *i* to vertex *j*, while an entry of 0 indicates that there is no directed edge from vertex *i* to vertex *j*. For example, row 0 indicates that vertex 0 has directed edges to vertex 1 and vertex 4. Furthermore, it can be observed that information about the edges incident on vertex 0 is not provided by this row of the adjacency matrix. For example, the information regarding the edge from vertex 5 to vertex 0 is provided by the row corresponding to vertex 5.

Now, let us see how the incidence matrix of a directed graph looks. To do this, add the following line of code at the end of the program called ‘*dgraph12.sagews*’.

```
DG.incidence_matrix( )
```

Upon executing the modified program, the following incidence matrix will also be displayed on the screen.

```
[-1 -1 0 0 0 0 0 0 1]
[ 1 0 -1 -1 0 1 0 0 0]
[ 0 0 1 0 -1 0 0 0 0]
[ 0 0 0 0 1 -1 -1 0 0]
[ 0 1 0 1 0 0 1 -1 0]
[ 0 0 0 0 0 0 0 1 -1]
```

The incidence matrix shown above represents the directed graph *Dgraph_12* with 6 vertices and 9 edges. Each row in the incidence matrix indicates a vertex, so there are 6 rows denoting vertices 0 to 5. Each column in the incidence matrix indicates an edge, so there are 9 columns denoting the 9 edges. In each column, an entry of -1 denotes the starting vertex and an entry of 1 denotes the ending vertex of that particular edge. For example, column 2 represents the edge from vertex 0 to vertex 4, which starts at vertex 0 (represented by -1) and ends at vertex 4 (represented by 1).

When we discussed undirected graphs, we came across connected graphs in which there are paths between any pair of vertices. There is a related concept in directed graphs called strongly connected directed graphs. A *strongly connected directed graph* is a directed graph in which there exists a directed path between any pair of vertices. Recall that, unlike undirected graphs, the existence of a path (directed) from vertex *A* to vertex *B* does not guarantee a path (directed) from vertex *B* to vertex *A*. An *orientation* of an undirected graph is a directed graph in which every original undirected edge is given one of the two possible orientations. A very interesting research topic in graph theory is the study of orientations of undirected graphs that result in strongly connected directed graphs—a topic of great interest to me, as my research in this area enabled me to obtain a PhD from IIT Palakkad.

Let us see how a strongly connected directed graph can be obtained from an undirected graph by introducing yet another highly useful SageMath method used to generate random graphs. A random graph is a graph where properties like the number of vertices, edges, and connections between them are determined randomly. In simpler terms, it is a graph generated by a random process, such as tossing a coin to decide whether to include an edge between a pair of vertices. Random graphs are used to model real-world networks such as social networks, biological networks, etc. Now, consider the code below, which uses the SageMath method called *RandomGNP()* to generate a random graph.

```
rgraph = graphs.RandomGNP(8, 1)
rgraph.show( )
```

Execute the above line of code several times on CoCalc and observe the graphs each time. You will see that the code always generates the same graph (an undirected complete